

Use Haskell to Access Two Popular Databases



Haskell, named after logician Haskell Curry, is a standardised, general-purpose, purely functional programming language, with non-strict semantics and strong static typing. In this ninth article on Haskell, we shall cover access to SQLite and MySQL databases using Haskell modules.

A number of packages are available from <https://hackage.haskell.org/packages/#cat:Database>, but, I will illustrate a few of them with examples. You first need to install the *cabal-install* tool on Fedora, for example, using the following command:

```
$ sudo yum install cabal-install
```

You can then install *HDBC.SQLite3* using the following command:

```
$ cabal install HDBC-sqlite3
```

This installs the latest 2.3.3.0 version from <https://hackage.haskell.org/package/HDBC-sqlite3>. You can also install the *SQLite3* package on Fedora for testing, as follows:

```
$ sudo yum install sqlite
```

To initiate a connection to a database, you can test it out

in the GHCi prompt using the *connectSQLite3* function, as shown below:

```
$ ghci
```

```
GHCi, version 7.6.3: http://www.haskell.org/ghc/?for help
Loading package ghc-prim ... linking ... done.
Loading package integer-gmp ... linking ... done.
Loading package base ... linking ... done.
```

```
ghci> :module Database.HDBC Database.HDBC.SQLite3
ghci> conn <- connectSQLite3 "students.db"
Loading package array-0.4.0.1 ... linking ... done.
Loading package deepseq-1.3.0.1 ... linking ... done.
Loading package old-locale-1.0.0.5 ... linking ... done.
Loading package time-1.4.0.1 ... linking ... done.
Loading package transformers-0.3.0.0 ... linking ... done.
Loading package bytestring-0.10.0.2 ... linking ... done.
Loading package text-0.11.3.1 ... linking ... done.
Loading package old-time-1.1.0.1 ... linking ... done.
```